

入る前に、TABPNSの確認(超ざっくり。)

背景：テーブルデータ対して、木ベースのモデルが主戦場であった。

ただ、課題:新しいデータセットが来るたびに、モデルを、一から、作り直す (fit) 必要がある。

具体的には、①ハイパーパラメータを、探索する(木の深さ、層数など) → ②その探索は、交差検証などで、何度も、試行錯誤する → ③ようやく、1つのモデルを、学習させる。

だからと言って、MLPは、表データに対して当てはまりが悪い。

そこで出てきたのが、TAB PNS_v1。

v1は、表データに対する、分類タスクが行える。行1つ1つをトークンとして扱い、attentionを与える。推論時には、データに対するアテンションは入れるが、パラメータの更新はしない。

v1での課題がいくつかあった。欠損値や意味のないデータがある場合。また、回帰問題にも対応できないなど。**v1=1,000** サンプルベース。。そこで、出たのがv2

v2は、行と列の双方向のattentionを採用。これにより、欠損値や意味のないデータがあっても精度が落ちなくなった。また、回帰問題に対しても、確率分布を出力することで対応可能。v2=10,000 サンプルベース。

TabICL

<https://arxiv.org/abs/2502.05564>

Abstract:

要約

(本文)

勾配ブースティング決定木の、長年の支配は、ICL(文脈内学習)を使う、表形式の基盤モデルによって、挑戦を受けている。TabPFNV2は、最大10,000サンプルの、表で、優れているが、列方向と行方向を、交互に行う、**attention**のために、大きな訓練セットを、扱うことが、計算上、法外に、高くつく。では、ICLを、効果的に、拡張して、より大きな表にも、恩恵を、もたらせるだろうか?私たちは、**TabICL**を、導入する。分類のための、表形式の基盤モデルで、最大60,000サンプルの、合成データセットで、事前学習され、手頃な、計算資源で、**50万サンプル**を、扱える。これを、可能にしたのは、**新しい、2段階のアーキテクチャ**である:行の、固定次元の、埋め込みを、構築する、**列→行の、attention機構**、それに続く、効率的な、ICLのための、Transformerである。TALENTベンチマークの、200個の、分類データセットで、TabICLは、TabPFNV2と、同等の性能でありながら、系統的に、速く(最大10倍)、他の、すべての手法を、大幅に、上回る。10,000サンプルを、超える、53個のデータセットでは、TabICLは、TabPFNV2と、CatBoostの、両方を、上回り、大規模データに対する、ICLの、可能性を、示す。

Introduction(要点):TabPFNV2は、列方向と、行方向を、交互に行う、2方向の

attentionを使う。これが、大規模データセットへの、拡張性を、制限する。実世界では、産業のデータセットが、数百万のサンプルを、含みうるので、TabPFNv2の、高い計算・メモリの需要は、実用性を、妨げる。

TabICLは、個々の、セルを、基本単位として、扱う(任意の、大きさの表に、対応するため)。2段階の、アーキテクチャ:第1段階で、行(ラベルを、除く)を、密な、ベクトル埋め込みに、符号化する。この段階が、**列の次元を、事実上、圧縮(collapse)**し、続く、ICLのための、計算量と、メモリを、大幅に、減らす。第2段階で、この、コンパクトな埋め込みを、対応する、ラベルと、組み合わせて、ICLを、行う。

さらなる、改良として:(1) TabPFNの、事前学習用の、合成データセットに、木ベースの、データ生成モデルを、新しく、加える。(2) カリキュラム学習を採用し、事前学習の、データセットサイズを、1,000から、60,000へ、段階的に、拡大する。(3) 10クラスを、超える、分類問題には、階層分類を使う。

(ポイント)

TabPFNv2は、すべての、12層で、m個の、列と、n個の、行の、両方について、attentionを、計算し続けます。一方、TabICLは、最初の段階でだけ、列と行を見て、そのあとは、**「1行=1つの、ベクトル」**に、まとめてしまいます。まとめたあとは、列の情報(m)は、もう、計算に、関わりません。行(n)どうしの、attentionだけが、残るので、mについての、計算が、大幅に、減る、というのが、直感的な、理由です。

↑: この部分が1ばんのポイントだけ、これから深く見ていきから、ポイントだけわかっていたらOK。

1. Introduction

言語モデルによる、表の扱い(第2段落)

(本文)

LLMは、主に、自然言語を、モデル化するために、設計されているが、表形式データのために、ファインチューニングする、試みも、現れている。これらの取り組みは、表の直列化(serialization)、つまり、表の行を、テキストや、文章に変換し、それを、トークン化する、過程に、頼っている。Gardner et al. (2024)は、直列化した表の、コーパスで、Llama 3-8Bを、ファインチューニングし、少数の例だけの、設定で、木ベースのモデルと比べた、この、アプローチの、有効性を、示した。しかし、こうした、言語モデルに基づく、アプローチは、大きな、直列化された、表を、収めるための、**文脈窓(context window)**の、大きさに、制限される(例えば、Gardner et al. 2024では、最大32、または、64ショットまで)。さらに、LLMが、数値を、効果的に扱えるかは、不明である。最後に、LLMが、多くの、人気のデータセットで、事前学習されている、という、証拠があるため、表形式の、予測タスクでの、評価は、注意して、行う必要がある。

特徴量埋め込みの、位置づけ(第7段落)

自然言語では、単語は、しばしば、明確な意味を持ち、それゆえ、自然に、情報豊富な、埋め込みと、結びつく。しかし、表形式データには、そうした、内在する構造が、ない:セルの値は、列名や、データ型といった、メタデータなしでは、あいまいになりうる。この課題に、取り組むために、TabICLは、(1) 各列内の、統計的な、規則性を、捉える、分布を意識した、列方向の埋め込みと、(2) 列をまたいだ、依存関係を、モデル化する、

attentionに基づく、行方向の相互作用を、組み合わせた、よく設計された、埋め込み戦略を、採用する。これにより、表形式データのための、意味的に、根拠のある、表現を、構築する。

Set Transformer の、導入 (第 8 段落)

特徴量の埋め込みは、各特徴量について、スカラーの、セルの値を、高次元の、ベクトルに、写像することを、含み、モデルの性能を、決める、重要な要因である。特徴量は、しばしば、大きく異なる分布を、示すため、これまでの、アプローチは、通常、パラメータの共有なしに、**特徴量ごとの、埋め込みモジュール**を、使っていたが、これは、表をまたいだ、転移可能性を、制限する。この研究では、私たちは、特徴量の埋め込みを、**集合を入力とする、問題**として、再構成する。ここでは、並べ替えに、不変な、セルの値の、集合が、入力として、働き、出力は、対応する、1対1の、埋め込みから、成る。これを、実現するために、私たちは、**Set Transformer**(Lee et al., 2019) を、活用する。これは、効率的な、induced self-attentionを通じて、集合を処理するために、特別に、設計された、モデルである。これは、極値(最大・最小)を、特定したり、一意な要素を、数えたりする、タスクで、優れており、各列内の、分布に関する、メタデータの、発見を、可能にし、異なる、データ型の、特徴量を、区別する、能力を、高める。

(ポイント)

第2段落=「なぜ、LLM(言語モデル)では、ダメなのか」

アプローチ	やり方	課題
LLM+直列化	表の、行を、 文章に、変換して、 Llamaなどに、読ませる	①文脈窓の、限界(せいぜい、32~64行)、②数値を、うまく、扱えるか、不明、③LLMが、そのデータで、事前学習済みかもしれない、という、不安
TabPFN/TabICL	表を、 文章に、しない。 セルの値を、そのまま、扱う	(この、課題は、生じない)

単語」と、「セルの値」の、対比

これは、なぜ、**Set Transformer**という、特別な、道具が、必要なのか、という、理由の、核心です。

- **言語:**「猫」という、単語は、**どの、文章に、出てきても、だいたい、同じ、意味を持つ。**だから、「猫」という、単語専用の、埋め込みベクトルを、1つ、あらかじめ、用意しておける (一般的な、言語モデルの、性質です)。
- **表:**ある、セルの値、たとえば、「**35**」という、数字は、**その列が、何であるかによって、意味が、まったく、違う。**年齢の、35かもしれないし、気温の、35かもしれない。しかも、****その列に、他に、どんな値が、入っているか(分布)****が、分からないと、「35は、平均的な値なのか、それとも、極端な値なのか」も、判断できない。

説明用の、例です。「35」という、値だけを、見せられても、それが、その列の中で、「一番、大きい方の値」なのか、「真ん中くらいの値」なのかは、**その列の、他の値も、見比べないと、わからない。**だから、TabICLは、「**その列の、値、全部の集合**」を、入力として、**Set Transformer**に、通し、「この、35は、この列の中では、こういう、位置づけ

だ」という、情報を、埋め込みに、反映させます。

Set Transformerが、得意な、タスクとして、「**極値 (最大・最小) を、特定する**」「**一意な要素の数を、数える**」が、挙げられています。

説明用の、例です。ある列に、[3, 7, 3, 9, 3]という、値があるとします。

- **最大値は、9(=極値)。**
- **一意な要素の、数は、3種類 (3、7、9)。**

こうした、「その列、**全体を、見渡さないと、わからない、統計的な、性質**」を、Set Transformerは、うまく、抜き出せる、という、主張です。これが、「distribution-aware(分布を、意識した)」の、意味です。この、能力が、Sec.3.2で、詳しく、使われることに、なります。

→ : Set Transformerの詳しいロジックは、今後出てくる。

2. Related Work

関連研究

Foundation Models and In-Context Learning

基盤モデルと、文脈内学習

(本文)

近年、深層学習は、基盤モデルの、登場によって、変革された。基盤モデルは、大規模で、多様なデータセットで、事前学習され、下流タスクのための、汎用的な、バックボーンとして、機能する。これらの、Transformerベースの、モデルは、文脈内学習 (ICL) を、可能にする:明示的な、訓練や、パラメータの更新なしに、入出力のペアを含む、プロンプトを、分析することで、タスクを、遂行する。ICLは、その場での、推論の、一形態として、働く。ICLの、根底にある、仕組みは、依然として、はっきりしておらず、有力な説明としては、暗黙のベイズ推論、勾配降下法による、最適化、アルゴリズム的な学習、といった、枠組みがある。

2.2. Tabular Deep Learning Models

2.2 表形式の、深層学習モデル

(本文)

CatBoost や、XGBoostのような、勾配ブースティング決定木 (GBDT) は、長らく、表形式の、領域を、支配してきた。しかし、表形式データのための、深層学習モデルを、開発する、努力が、高まっている。最近の研究は、GBDT と、表形式の、深層学習モデルの、性能差が、狭まっていることを、示している。

表形式の、深層学習が、改善するにつれて、表をまたいだ、転移可能性が、重要な、話題として、浮上している。この方向の、注目すべき、取り組みには、XTab や、CARTE が、含まれ、これらは、**転移可能な、コンポーネント** (通常、共有可能な、バックボーンネッ

トワーク)と、**データセット固有の、コンポーネント**(新しいタスクごとに、ファインチューニングが、必要)を、組み込む。表形式の、基盤モデルの、登場は、表をまたいだ、学習に、新しい可能性を、もたらしうる。表全体にわたる、大規模な、事前学習と、転移学習への、道を、開くものである。

2.3. Tabular Foundation Models

2.3 表形式の、基盤モデル

(本文)

TabPFN(Tabular Prior-Data Fitted Networkの、略)は、表形式の、基盤モデルである。これは、広範な、合成データセットで、事前学習された、Transformerであり、ICLを通じて、表形式の、分類タスクを、行う。TabPFNは、ICLを、ベイズの、視点から、合成データセットにわたる、近似的な、事後予測分布として、解釈する。いくつかの、派生形が、その、拡張性を、高めることを、目指している。訓練データを、プロンプトチューニングを通じて、コンパクトな、学習済みの、文脈に、蒸留すること、各、テストサンプルについて、最も、関連性の高い、訓練データの、部分集合を、選ぶこと、2次のattentionを、線形のattentionに、置き換えること、ICLに基づく、ハイパーネットワークを通じて、小さな、タスク特化の、ニューラルネットワークを、生成すること、などである。しかし、ほとんどの、派生形は、TabPFNを、**構造的に、改善するのではなく**、文脈内の、サンプルを、減らすための、**プロンプトエンジニアリング**として、働いているに、過ぎない。

いくつかの、TabPFNの派生形は、事前学習データの、質を、改善しようとしている。たとえば、TabForestPFNは、木ベースの、合成データセットを、組み込み、TabDPTは、実世界のデータセットを、使う。

TabPFNV2は、予測性能と、拡張性の、両方の点で、TabPFNを、大幅に、改善する。私たちの、新しいモデル、TabICLは、TabPFNV2に、匹敵する、性能を、達成しながら、より、拡張性が高く、計算効率が良い。付録Aは、アーキテクチャ、事前学習、拡張性にわたる、TabPFNV2と、TabICLの、体系的な比較を、提供する。

(ポイント)

Sec.2の、位置づけ

- ・この節は、**先行研究の、まとめ**です。TabICL、自体の、新しい主張は、少なく、「これまで、誰が、何を、やってきたか」を、整理しています

- ・これまで、TamPNSの派生毛として、プロンプトチューニングけいや、部分集合系、NN系統などが考慮されたが、著者らは、はっきりと、「ほとんどの、派生形は、**構造的な、改善では、ない**」と、述べています。つまり、TabICL、自身は、**「単なる、工夫」ではなく、「アーキテクチャそのものの、作り直し」**である、という、**自負**が、この、一文に、込められています(AIの読みです)。

- ・XTabや、CARTEは、表をまたいだ、転移可能性(cross-table transferability)**という、テーマの、代表例として、この、2つが、挙げられています。ここで転移可能性とは、表Aで、学んだことを、まったく、別の、表Bにも、使い回せる、能力。

3. The TabICL Architecture

(本文)

Sec.3 の、冒頭

私たちは、入力空間 $X \in \mathbb{R}^m$ と、目的空間 $Y \in [1, \dots, C]$ を持つ、表形式の、分類タスクを、考える。入出力の、ペアの、訓練データセット $D_{\text{train}} = \{(x_{\text{train}}^{(i)}, y_{\text{train}}^{(i)})\}$ ($i=1 \sim n_{\text{tr}}$) と、テストサンプル x_{test} が、与えられたとき、私たちの、目標は、クラス確率 $p(\cdot | x_{\text{test}}, D_{\text{train}})$ を、予測することである。

3.1 全体構造:埋め込みしてから、ICL

TabICL は、2つの、主要な、モジュールから、成る:**表の、埋め込みモジュール**、それに続く、**ICL モジュール**。ラベルは、**ICL モジュールでのみ**、使われる。表の、埋め込みモジュールは、行と列の、内在する構造を、考慮しながら、表の行を、密なベクトル表現に、符号化する。このモジュールは、2つの、構成要素から、成る:**各、列の、統計的な、特性を、捉える、分布を意識した、列方向の埋め込み**と、**特徴量間の、依存関係を、モデル化する、文脈を意識した、行方向の相互作用**である。続いて、ICL モジュールは、訓練と、テストの、両方の、埋め込みを、Transformer を通じて、処理し、ICL を通じて、テストセット全体の、予測を、単一の、順伝播で、可能にする。全体の、アーキテクチャは、図1に、示す。

3.2 分布を意識した、列方向の埋め込み

列方向の、埋め込み(または、特徴量埋め込み)は、列 $c_j \in \mathbb{R}^n$ の中の、各、スカラーの、セルを、 d 次元の、埋め込みに、写像する。各、列に、別々の、埋め込みモジュールを、割り当てる、典型的な、アプローチとは、違い、私たちは、**共有可能な、Set Transformer TF_col** を通じて、すべての、列を、埋め込む。図2bに、示し、以下のよう、に、定式化する:

$$W, B = \text{TF_col}(c_j) \in \mathbb{R}^{(n \times d)} \quad (\text{式1})$$

$$e_j = W \odot c_j + B \in \mathbb{R}^{(n \times d)} \quad (\text{式2})$$

各、列の中の、各セルには、それぞれ、独自の、重みと、バイアスが、割り当てられる、ことに、注意されたい。本質的に、特徴量埋め込みは、**集合入力の、問題として、捉えることができる**、TF_col は、並べ替えに、不変な、セル値の、集合を、入力として受け取り、**分布を意識した、重み W と、バイアス B を、生成する、ハイパーネットワークとして、働く。**TF_col の、内部の、演算は、以下のように、展開される:

$$U = \text{Lin}(c) \in \mathbb{R}^{(n \times d)} \quad (\text{式3})$$

$$M = \text{MAB1}(V_I, U_{\text{train}}, U_{\text{train}}) \in \mathbb{R}^{(k \times d)} \quad (\text{式4})$$

$$V = \text{MAB2}(U, M, M) \in \mathbb{R}^{(n \times d)} \quad (\text{式5、この2つが、ISAB})$$

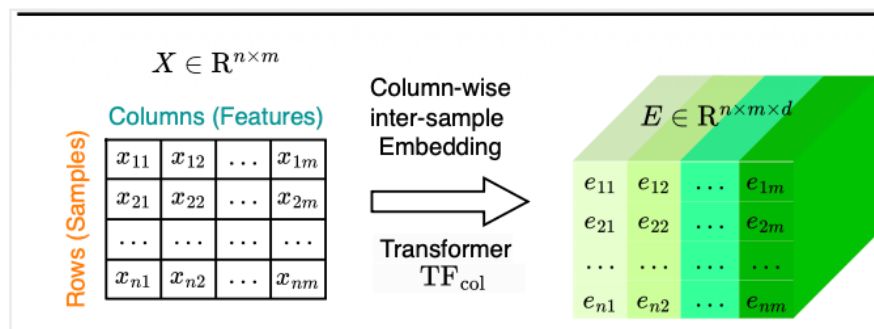
$$W, B = \text{Lin}(V) \in \mathbb{R}^{(n \times d)} \quad (\text{式6})$$

ここで、Lin は、線形層、MAB は、マルチヘッド・アテンションブロック、ISAB は、Lee et al. (2019) が、導入した、**誘導 self-attention ブロック**である。列 c (簡単のため、添字 j を、省略) は、線形層を通じて、 d 次元の、空間 ($d=128$) に、射影され、MAB1 と、MAB2 から、成る、ISAB で、処理され、線形層を通過して、 W と B を、生成する。

ISAB は、self-attention の、複雑さを、 **$O(n)$** に、減らしながら、**2段階の、attention 機構**を通じて、大域的な、情報を、捉える、能力を、保つ。MAB1 では、**誘導ベクトル V_I** が、クエリとして働き、訓練サンプル U_{train} に、attention して、誘導され

た、表現 M を、生成する。MAB2では、入力 U (訓練と、テストの、両方の、サンプルを含む) が、クエリとして働き、 M に、再び、**attention** し、大域的な、情報が、すべての、訓練サンプルに、伝わることを、可能にする。私たちは、 **$k=128$ 個の、誘導ベクトル**、MABの中の、**4つの、attention head**、**3つの、ISABブロック**を、使う (明確さのため、式4と、5には、1つの、ISABブロックだけを、示している)。決定的に、**MAB1**では、**訓練サンプルだけが、キーと、値として、働く**。これにより、ISABの、出力が、**訓練データだけに、依存することが、保証され、データ漏洩を、防ぐ**。

学習された、特徴量埋め込みが、何を、捉えているかへの、洞察を、得るため、私たちは、TF_colの、最後の、ISABブロックの、出力である、 $M \in \mathbb{R}^{(k \times d)}$ (式4) を、可視化する。私たちは、 M を、**第1次元に沿って、合計する** (すなわち、すべての、誘導ベクトルの、誘導された、表現を、集約する) ことで、**列ごとに、1つの、ベクトルに、要約し**、それから、****主成分分析 (PCA)****を、適用する。図3は、**似た歪度 (または、尖度) を持つ、列が、まとまる傾向を、明らかにする**。すなわち、**非対称な、分布は、対称な、分布と、区別され (図3、左)、裾の重い、分布は、裾の軽い、分布と、区別される**。これは、TF_colが、分布の、性質を、構造化された、形で、符号化しており、列の中の、セルは、おそらく、**その、統計的な、役割** (たとえば、最小、最大、平均、最頻値) を、反映するように、埋め込まれていることを、示唆する。したがって、学習された、特徴量埋め込みは、特徴量を、その、独自の、分布の性質に基づいて、**区別するはずであり、事実上、特徴量の、識別子として、働く**。これは、意味的な、列名に、頼る、方法 (CARTE) や、特徴量の、識別子ベクトルを、学習する、方法とは、**対照的である**。



(ポイント)

やってること：

ここで数式が出てくるが、今回の部分で扱っているのは、表の入力があったとき、まず列方向の処理の部分。

パーツ：

記号	たとえでの、何か	実際の、形
c	1つの、列の、生の値、n個(=たとえの、10,000人の、生データ)	R^n (n個の、数字)
U	cを、ベクトルに、変換したもの(=10,000人を、まず、ベクトルの、名札に、変える)	$R^{(n \times d)}$
V_I	誘導ベクトル(=128人の、代表)。学習される、固定の、初期値	$R^{(k \times d)}$ 、 k=128
M	代表が、作った、要約	$R^{(k \times d)}$
V	全員が、その、要約を、見て、更新した、自分の、ベクトル	$R^{(n \times d)}$

数式の大枠：
 表のセルの値をcとして、cを使って、 $W \cdot b$ を出力(式：3,4,5,6,)。その、 $W \cdot b$ をcにかけて足してる(式：1,2)。
 具体的に、どのようにして、wbを出力しているのか。

U=Lin(c) — 「まず、名札を、作る」

$U = \text{Lin}(c) \in R^{(n \times d)}$
 たとえでの、位置づけ:これは、「128人の、代表を、立てる、前段階」です。10,000人の、**生の、数字(c)**を、まず、**d次元の、ベクトル(名札)**に、変換します。「Lin」は、線形層(前に、確認した、エンコーダと、同じ、役割です)。
 説明用の、数値です。ある、セルの値が、c=5.0だとします。線形層を、通すと(数式は、省略して、結果だけ)、たとえば、 $u=0.2, -0.5, 1.1, \dots$ に、なります。**これを、n個の、セル、すべてに、行うので、U全体は、 $n \times d$ の、表に、なります。**
 → 次:式(4)です。

ステップ3:式(4) M=MAB1(V_I, U_train, U_train) — 「128人の、代表が、訓練データを、見て、要約を、作る」

$M = \text{MAB1}(V_I, U_{\text{train}}, U_{\text{train}}) \in R^{(k \times d)}$
 これが、まさに、たとえの、**ステップ1**「128人の、代表が、10,000人(訓練データだけ)を、見て、要約を、作る」、です。
MABの、括弧の中の、3つの、引数は、"誰が、誰を、見るか"を、表しています。前に、確認した、attentionの、たとえ(Query・Key・Value)を、思い出してください。**MAB**の、引数は、**(Query、Key、Value)**の、順です。

引数	ここでの、中身	役割
1つ目:Query	V_I(128個の、誘導ベクトル)	「見る側」=代表
2つ目:Key	U_train(訓練データの、名札)	「見られる側の、目印」=訓練データ
3つ目:Value	U_train(同じ、訓練データ)	「見られる側の、中身」=訓練データ

つまり、式(4)は、「**128個の、代表(V_I)が、Queryとして、訓練データ(U_train)に、attentionし、その結果を、Mとして、受け取る**」という、意味です。
 説明用の、数値です。128個の、代表のうち、1つ(V_Iの、1行目)が、訓練データの、

10,000個の、 u (名札)を、見て、「近い、名札を持つ、データほど、重視して、平均する」(前に、確認した、softmax+重み付き平均、そのものです)。その、結果、できた、1個の、ベクトルが、 M の、1行目に、なります。これを、128個、繰り返すと、 M は、**128×d**の、表に、なります。

****「 M =要約」****という、たとえば、ここで、式と、完全に、一致します。

→ 次:式 (5) です。

ステップ 4:式 (5) $V=MAB2(U,M,M)$ — 「全員が、要約を、見て、自分の、位置を、確認する」

$V = MAB2(U, M, M) \in R^{(n \times d)}$

これが、たとえの、**ステップ 2**「10,000人、全員が、128人の、要約だけを、見て、自分の、位置を、確認する」、です。

同じく、(Query、Key、Value)で、見ます。

引数	ここでの、中身	役割
1つ目: Query	U (訓練+テスト、全員の、名札)	「見る側」=全員
2つ目: Key	M (128個の、要約)	「見られる側の、目印」=要約
3つ目: Value	M (同じ、要約)	「見られる側の、中身」=要約

つまり、式 (5) は、「**全員 (U) が、Queryとして、128個の、要約 (M) に、attention し、その結果を、 V として、受け取る**」という、意味です。

説明用の、続きです。テストの、あるデータ(名札 u_{test})が、「128個の、要約 (M) の、どれに、近いか」を、見比べ、「近い、要約ほど、重視して、平均する」ことで、**自分の、新しい、ベクトル v_{test}** を、得ます。

重要な、点:ここでは、**Query(見る側)**は、訓練も、テストも、全員、参加しています。しかし、**見られる側 (Key、Value)**は、 M であって、他の、生データでは、ありません。だから、テストのデータが、直接、他の、テストデータや、訓練データの、生の値を見ることは、なく、必ず、 **M** という、"**フィルターを、通して**"、しか、情報を、得られません。これが、計算量を、抑える、しかけです(全員×全員= n^2 ではなく、全員×128= $n \times 128$ に、なるから)。

→ 次:式 (6) です。

ステップ 5:式 (6) $W,B=Lin(V)$ — 「要約結果を、レシピに、変換する」

$W, B = Lin(V) \in R^{(n \times d)}$

V は、まだ、「その、セルが、要約の中で、どんな、位置づけか」を、表す、ベクトルです。これを、もう一度、線形層に、通して、****「このセルに、掛ける、重み W 」と、「足す、バイアス B 」****という、**2つの、数字の組**に、変換します。これが、式 (1)、(2) で、確認した、「レシピ」です。

確認：

れまでに、出た、式 (1) ～ (6) は、すべて、TF_col(列方向) だけ

前回までに、確認した、式は、すべて、****3.2節 (Distribution-aware Column-wise Embedding)****の中に、あるもので、**TF_col**という、**1つの、Transformerの、中身**を、説明する、ためだけの、ものでした。

(補足)

・「attentionを、適用する」には、実は、**2つの、段階が、**あります。

段階	やること
段階1:QKVを、用意する	入ってきた、ベクトルから、Q(クエリ)、K(キー)、V(バリュー)という、3種類の、ベクトルを、作る
段階2:QKVを、使って、計算する	QとKの、関連性(スコア)を、計算し、それを使って、Vを、重み付き平均し、新しい、ベクトルを、出す

・誘導ベクトル V_I は、 θ の一部のため、学習される。

3.3. Context-aware Row-wise Interaction

(本文)

すべての、特徴量の、埋め込み、 $E \in \mathbb{R}^{(n \times m \times d)}$ を、得たあと、**3層、8ヘッドの、Transformer(TF_row)**が、Eを処理し、特徴量間の、相互作用を、扱う。埋め込みを、1つのベクトルに、集約するため、**4つの、学習可能な、[CLS]トークン**が、各行の、先頭に、付け加えられ、その、最終的な、出力が、**連結**される。4つのトークンを、使うのは、続く、ICLのために、 **$4 \times d = 512$** という、豊かな、表現を、提供するためであり、一方で、TF_colと、TF_rowでは、 **$d = 128$** という、低い、埋め込みサイズを、保つことで、メモリの、消費を、抑える。

私たちは、実験的に、TF_rowが、**表現崩壊**の、問題に、苦しむることを、観察した。前述のとおり、特徴量は、列方向の、埋め込みのあと、その、**分布の性質**によって、識別される。その結果、**似た、分布から、来る、特徴量は、区別しにくくなる**。すべての、特徴量が、同じ、分布から、引かれる、極端な場合では、素の、並べ替え不変な、self-attentionを使う、TF_rowは、あるサンプルを、その、**列を、並べ替えた、どのバージョンとも、区別できず**、元々は、異なっていた、サンプルが、ほぼ、同一の、表現に、なってしまう。これが、**表現崩壊**である。この、現象は、**balance scale**データセットで、例証される(図4)。このデータセットでは、すべての、特徴量が、5つの、値しか、取らない、同じ、離散分布に、従うため、崩壊が、起きやすい。TF_rowで、処理したあと、多くの、サンプルが、同じ、表現に、崩壊する(一番右の、プロット)ことが、観察されるが、元々は、異なっていた(一番左の、プロット)。

TabPFNv2でも、同様の、表現崩壊の、問題が、報告されている。これを、緩和するため、**Hollmann et al. (2025)**は、各、特徴量に、ランダムな、特徴量識別子ベクトルを、導入し、特徴量の、**グループを、個別ではなく、集合的に、符号化**した。対照的に、私たちは、異なる、戦略を、採用する。TF_rowに、**RoPE(回転位置埋め込み)**を、組み込み、同一に、分布する、特徴量間の、対称性を、崩す。RoPEは、最近のLLMで、広く、採用されており、クエリと、キーの、ベクトルを、**回転させる**ことで、相対的な、位置情報を、attention機構に、直接、符号化する。回転の、角度は、系列中の、位置 p と、次元の、index i によって、決まり、 $\theta_i = p / (\text{base}^{(2i/d)})$ と、定義される。 d は、埋め込みの、次元、baseは、周波数の、スケーリング係数である。

RoPEの、使用は、**列の順序に関する、並べ替え不変性を、崩すが**、表現崩壊を、効果的に、緩和できる。たとえば、balance scaleデータセットでは、RoPEは、異なる、サン

プル間で、異なる、表現を、保つ (図 4、中央)。私たちは、**100,000** という、大きな、スケーリング係数を、使い、訓練で、見た (最大 100 個) より、多くの、特徴量への、汎化を、高める。並べ替え不変性を、近似的に、回復するため、TabICL は、他の、**TabPFN** 類似のモデルと、同じ、戦略を、採用する。すなわち、複数の、列の並べ替えにわたって、予測を、アンサンブルする。

3.4 節

すべての、サンプルを、埋め込み $H \in \mathbb{R}^{(n \times 4d)}$ に、変換したあと、訓練ラベルは、**one-hot** エンコーディングを使って、 H と、同じ、空間に、写像される。訓練セットについての、 X と、 y の、埋め込みは、**足し合わされ**、最終的な、訓練の、埋め込み、 H_{train} を、作る。それから、私たちは、**12 層、4 ヘッドの、Transformer(TF_icl)** を使って、 H_{train} と、 H_{test} を、処理する。

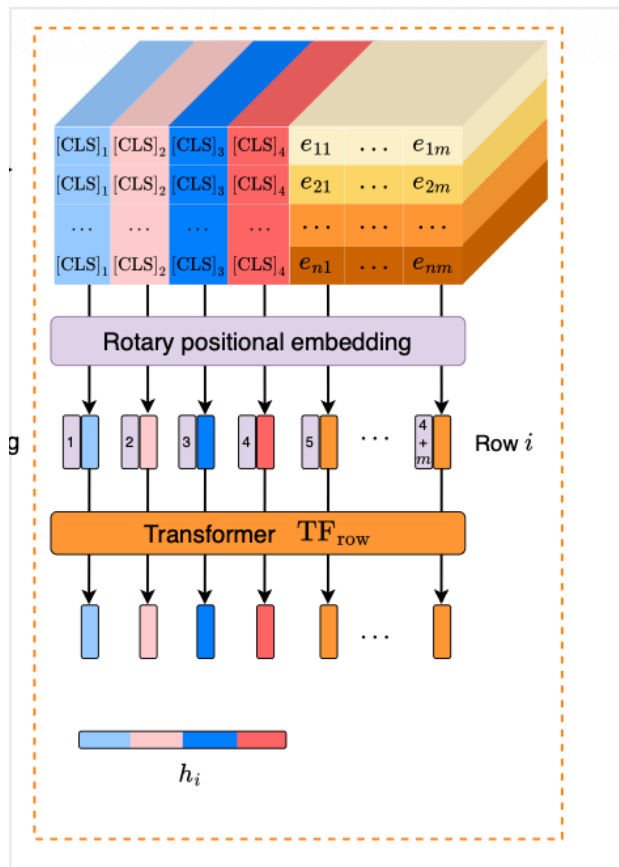
3.5 節

n 行、 m 特徴量の、表について、3 つの、構成要素の、時間計算量を、分析する。

1. **TF_col**: ISAB を使って、各、列を、独立に、処理し、複雑さは、 **$O(nkm)$** 。 k は、固定の、誘導ベクトルの、数。
2. **TF_row**: 各、行につき、特徴量と、4 つの、[CLS] トークンに対して、self-attention を、適用し、複雑さは、 **$O(m^2n)$** 。ほとんどの、表形式の、設定では、 $m \ll n$ なので、TF_row は、効率的。
3. **TF_icl**: 圧縮された、行の、埋め込みに対して、self-attention を、行い、複雑さは、 **$O(n^2)$** 。 n について、2 次だが、**特徴量の、次元が、圧縮されているので**、このコストは、**一度だけ**、発生し、実用上の、効率性を、確保する。

TabICL の、全体の、複雑さは、 **$O(m^2n + n^2)$** 。対照的に、TabPFNV2 は、次元を、圧縮せずに、列方向と、行方向の、attention を、交互に行い、複雑さは、 **$O(m^2n + n^2m)$** 。結果として、TabPFNV2 は、大きな n と、中程度の m で、計算コストが、高くなりうる。

(ポイント)



ここでやってること：

TFrow（行方向のアテンションの処理）の詳細をやっている。図1の右側の部分。

- ①：各行に対して先頭に[CLS]を4つ付ける。[CLS]は学習可能な特殊なトークン。このイメージは、各ベクトルをどのような目的（例：ベクトル自体が食べ物だとしたら、どこ産か？、値段は？）などの役割を果たす。これにより、もともと、TFZcolを経て出てきたベクトルが128次元だったのに対して、4倍の512次元で表現することができる。
- ②：次に、RoEBを実装。これは、transformerの最初のポジショナルエンベディングと同じ。sinとcosを使って、ベクトルを変換させる。これを実装する目的は、TFcolによって、各特微量別に、それぞれの表のその特微量が、どのような値であるかを、確率分布で出力している。そのため、似たような値を取る特微量は、似たような分布となる。（図3・4がその可視化した結果。）それにより、分布として扱えるが、分布は同じでも、特微量自体が、年収なのか信用度であるのかといった、本来の特性が失われてしまう。そこで、PoPEを採用することで、区別の1基準となる。ただ、それとトレードオフで、表示したいの不変性（順番を入れ替えても、対応できる）といった、区政が失われる。
- ③：次に、[CLS]とPoEBを経たベクトルに対して、TFrowを行う。ここでは、TFcolのそう、誘発ベクトルは組み込まれていない。一般的なtransformer。attentionの見る方向は、行同士。
- ④：最後にその出力が、[CLS]の流れで、4つになっているため、それを直列にくっつけると h となる。

誘発ベクトルを使わない理由：

Transformer	計算量	nと、mの、どちらが、多いか
TF_col	$O(nkm)$	n(行数)は、非常に、大きいことが、多い(たとえば、100,000行)
TF_row	$O(m^2n)$	m(列数)は、比較的、小さい(たとえば、100列まで)

TF_colが、見ている、"n"(行の、数)は、非常に、大きく、なりうるので、普通の、self-attentionを、使うと、 n^2 で、計算が、爆発してしまいます。だから、誘導ベクトルという、"近道"が、必要でした。

一方、TF_rowが、見ている、"m"(列の、数)は、せいぜい、数百です(本文にも、「訓練で、見た、最大、100個の、特徴量」と、ありました)。数百程度なら、普通の、self-attentionでも、計算量は、そこまで、問題に、ならないので、誘導ベクトルのような、近道は、不要、と、考えられます(私の、読みです)。

↓

4. Pretraining and Inference

(本文)

4.1節 (前段:SCM)

TabICL は、合成データセットだけで、事前学習される。変数間の、現実的な、依存関係を、確保するため、私たちは、**TabPFN(v1)** で、提案された、アプローチに、従って、SCMを使って、データセットを、生成する。まず、**DAG**を、サンプリングして、依存関係を、定義する。これは、完全連結の、MLPの、構造に、従い、各、ニューロンが、1つの、変数に、対応する。各、特徴量cは、そのグラフの中の、親の、変数、**Pa(c)**の、関数**f**として、モデル化され、独立の、ノイズ**ε**が、加わる ($c=f(Pa(c))+\epsilon$)。先行研究と比べて、私たちは、2つの、方法で、データセットの、生成を、豊かにする:(i) **木ベースの、SCM**を、導入し、その、帰納バイアスの、恩恵を、受ける。(ii) **モデル化する、関数fの、多様性を、増やす**。データ生成の、バランスを取るため、**SCM(70%)と、木ベースのSCM(30%)**を、組み合わせる。

木ベースの、生成:Grinsztajn (2024) に、着想を得て、私たちは、変数間の、複雑な、相互作用と、階層的な、依存関係を、モデル化する、能力を、活かすため、木ベースのSCMを、導入する。fを、**XGBoost**の、**回帰モデル**を使って、定義する。実務者に、広く、好まれ、多出力の、回帰を、サポートしているからである。グラフの、各層で、ガウスノイズから、引いた、**"偽の、目的変数"**で、XGBoostモデルを、訓練し、親の、変数の、値を、入力として、取る。得られた、予測が、**子の、変数の、値に、なる**。

活性化関数の、多様化:TabPFN(v1)では、fは、[Identity、Tanh、Leaky ReLU、ELU]から、選ばれた、活性化関数を、持つ、**ランダムな、アフィン写像**として、定義される。私たちは、これを、**15個の、追加の、活性化関数**で、豊かにし、非単調、または、不連続な、関数を、例として、導入する。私たちは、また、**ランダムな、カーネル**を持つ、**ガウス過程**から、**サンプリングされた、ランダムな、活性化関数**も、含める。最後に、層ごとに、違う、活性化関数を、使う、オプションを、加え、各、活性化関数の、前に、**標準化と、ランダムな、再スケーリング**を、適用した。

4.2節

LLMを、短い、文から、事前学習し、その後、長い、文に、移るのと、似た、方法で、

私たちは、合成データセットの、大きさ(サンプル数)を、段階的に、増やしながら、**マイクロバッチサイズ N_B** を、調整し、メモリの、制約に、対応する。**3段階の、手順を、採用した:**

1. $N_B=4$ 、**固定サイズ1,024**で、**160,000**ステップ。
2. $N_B=1$ 、サイズは、**1,000~40,000**の間の、対数一様分布から、ランダムに、引く。**2,000**ステップ。10,000サンプルを、超える、データセットには、アクティベーションのチェックポイントニングを、有効にし、それに、応じて、特徴量の数を、減らし、メモリ不足を、避ける。
3. $N_B=1$ 、サイズは、**40,000~60,000**の間で、一様に、サンプリング。**50**ステップ。**TF_icl**だけを、訓練し、他の、すべての、構成要素は、凍結する。

各、ステップは、512個の、データセット(特徴量、100以下、クラス、10以下、ランダムに、サンプリング)から、成る。事前学習は、**3つの、A100 GPU(40GBメモリ)**で、**20日間**、要した(段階、1、2、3で、それぞれ、16日、3日、1日)。

4.3 節

私たちは、**多クラス(10クラスを、超える)**の、分類問題に、**階層分類**を通じて、取り組む。具体的には、クラスを、**最大10クラスの、部分グループ**に、再帰的に、かつ、均等に、分割し、**多段階の、分類木**を、形成する。 k クラスの、分類問題には、深さ、 $r=\lceil \log_{10} k \rceil$ の、階層が、必要である。木の中の、各、内部ノードは、その、**子グループの、確率**を、予測し、各、**葉**は、それが、含む、クラスの、確率を、予測する。推論の間、ある、クラスの、最終的な、確率は、その、クラスに、至る、唯一の、**根から、葉までの、経路上の、すべての、ノードにわたる、確率を、掛け合わせる**ことで、得られる。前述のとおり、ラベルは、**最終的な、ICLブロックでのみ**、使われる。したがって、階層木は、**データセット単位のICLの、間に、構築され、すべての、サブタスクは、同じ、学習された、行の、埋め込み H と、同じ、TF_iclを、ICLの、予測のために、共有する**。この、種類の、共有は、階層分類の、場面で、TabICLの、効率性を、大幅に、高める。

4.4 節

FlashAttentionを、使うことで(系列の、長さに対して、線形の、メモリの、複雑さを、提供する)、私たちは、**ピークの、活性化メモリ**が、推論の、間、**多項式回帰**で、効果的に、モデル化できることを、観察した: $\alpha_1 \times \text{バッチサイズ} + \alpha_2 \times \text{系列長} + \alpha_3 \times \text{バッチサイズ} \times \text{系列長} + \alpha_4$ 。これにより、系列の長さ、と、利用可能な、GPUメモリに、基づいて、**バッチサイズを、動的に、調整**できるように、なった。バッチの、次元は、文脈によって、違う、役割を、果たす:**列方向の、埋め込みでは、列の、数を、行方向の、相互作用では、サンプルサイズを、データセット単位のICLでは、データセットの、数を、表す**。

Rajbhandari et al. (2021) に、着想を得て、**中間の、活性化を、必要に応じて、CPUや、ディスクに、退避させ**、さらに、メモリの、消費を、減らせる。これらの、最適化により、TabICLは、**わずか、5GBの、GPUメモリと、32GBの、RAMで、100,000サンプル、500特徴量の、データセットを、扱えるように、なる**。これは、多くの、実世界の、応用に、十分である。

(ポイント)

SCMの拡張

拡張	内容
(i) 木ベースのSCM	fを、XGBoostで、定義する、新しい、種類。70%は、通常のSCM、30%は、木ベース、という、混合比率も、明記
(ii) 活性化関数の、多様化	v1の、4種類([Identity、Tanh、Leaky ReLU、ELU])から、+15種類、さらに、ガウス過程による、ランダムな、関数も、追加

カリキュラム学習=「小さい表→大きい表へ、段階的に：
扱うデータが多く区なるにつれ、バッチサイズ（一度に処理する、列・行・表）の数が減少している点がポイント

段階	サイズ	ステップ数	特徴
1	固定、1,024	160,000	v1と、同じ、規模で、しっかり、練習
2	1,000~40,000(ランダム)	2,000	大きな、表にも、慣れる。メモリ節約の、工夫を、導入
3	40,000~60,000	50	TF_iclだけを、訓練。他は、凍結

当たり前だが、扱う表が大きくなれば、行すうが増えるので、一度に処理できるバッチ数は減少する。

隠れたポイント：
前々回、3.1節で、確認した、「ラベルは、ICLブロック (TF_icl) でのみ、使われる」という、設計が、ここで、活かされます。TF_col、TF_rowの、出力 (H) は、クラスの、数に、関係なく、1回だけ、計算すれば、済みます。すべての、サブタスク (1段目、2段目...) が、同じ、Hと、同じ、TF_iclを、使い回すので、Hを、何度も、作り直す、必要が、ないわけです。
もし、ラベルが、TF_colや、TF_rowの、時点で、混ざっていたら (v2のような、"早期融合")、サブタスクごとに、別々の、埋め込みを、作り直す、必要が、あったかもしれません。前々回、確認した、付録Aの、**「TabICLは、後期融合、TabPFNV2は、早期融合」**という、違いが、ここで、実際の、利点として、生きています。

5. Experiments

(本文)

5.1節

私たちは、Ye et al. (2025) が、導入した、TALENTベンチマークで、TabICLを、評価する。これは、200個の、分類データセット (120個の、二値、80個の、多クラス) か

ら、成り、30を超える、ベースライン手法の、結果が、付属する。公平な、比較を、確保するため、**TabPFNV2が、ハイパーパラメータ調整と、モデル選択に、使った、15個のデータセットを、除外する。**以下では、主に、**10クラス以下の、171個のデータセット**に、焦点を当てる。

データセットは、****訓練64%、検証16%、テスト20%****に、分割される。ほとんどの、モデルは、**早期終了と、ハイパーパラメータ調整のために、検証セットに、頼るが、TabICLと、TabPFNV2は、そうしない。**それでも、私たちは、**訓練データだけ**を使って、TabICLと、TabPFNV2を、学習させることを、選んだ。これは、**両者を、不利な、立場に、置く。**一方で、TabICLと、TabPFNV2の、両方は、**アンサンブルを、活用する**(他の、深層学習モデルとは、違う)。

TabICLと、TabPFNV2の、両方は、**列と、クラスを、ランダムに、並べ替え、異なる、前処理を使って、得た、32回の、予測を、平均する。**TabICLは、**べき乗変換の、有無で、z正規化を、すべての、入力特徴量に、適用する。**

5.2節

TabICLは、最先端の、正解率を、達成する。図5は、**すべての、モデルの、調整済みMLPに対する、相対的な、正解率を、示す。**TabICLは、**すべてのデータセットにわたって、最良の、中央値の、相対的な、正解率を、達成しながら、従来の、最先端モデルより、はるかに、速い:TabICLの、1,000サンプルあたりの、学習+推論の、相乗平均時間は、1.1秒、CatBoostを、CPUで、調整するには、約3分、RealMLPと、ModernNCAを、GPUで、調整するには、約7分、かかる。**TabICLと、TabPFNV2は、**大差で、競合を、上回るが、両者の、差は、統計的に、有意では、ない。**

TabPFNV2への、速度向上。図6は、**TabICLが、小さな、データセットで、1.5倍、大きな、データセットで、3~10倍、速いことを、示す。****10,000サンプル、100特徴量の、データセットでは、TabICLは、約20秒、対して、TabPFNV2は、1分40秒。****1,000サンプル、10特徴量では、TabICLは、1秒、対して、TabPFNV2は、2秒。**この、実験的な、速度向上は、Sec.3.5の、計算量の、分析を、裏付ける。**平均的な、速度向上は、大きなデータセットで、約、5倍に、留まる。**

TabICLは、大きな、データセットでの、ICLを、可能にする。TabPFNV2は、**10,000サンプルまでの、データセットで、優れた、性能を、達成するが、最大2,048の、訓練サンプルで、事前学習されただけであり、そのメモリ使用のために、30,000サンプルを、超えると、失敗しうる。**図7は、TabPFNV2とは違い、TabICLの、性能は、より、大きな、データセットでも、**強いままであることを、示す。**

****TabICLは、信頼できる、確率を、生成する。******TabICLは、ハイパーパラメータ調整を、活用しないので、その、予測は、正解率だけに、特化して、最適化されていない。**付録Iの、**臨界差の図は、TabICLと、TabPFNV2が、log lossにおいて、正解率に、調整された、競合を、大きく、上回ることを、示すが、2つの、差自体は、有意では、ない。**

****TabICLは、10クラスを、超えても、有効である。******10クラスを、超える、データセットでは、私たちは、TabICLに、Sec.4.3の、階層分類の、戦略を、適用する。**ラベルに、依存しない、行の埋め込みを使い、これを、すべての、サブ分類器の間で、共有できることのおかげで、TabICLは、多数のクラスに、効率的に、スケールする。図8は、**TabICLが、これらの、データセットで、平均、正規化正解率で、依然として、2位を、達成することを、示す。**一方、TabPFNV2は、**10クラスを、超える、ものを、そのままでは、扱えない。**

5.3節

****木ベースのSCMは、有益であると、証明される。******事前学習の間、木ベースのモデルの、帰納バイアスを、組み込むことは、TabICLの、性能を、改善する(図9)。**

****カリキュラム学習は、大きなデータセットでの、TabICLの、性能を、効果的に、改善**

する。****カリキュラム学習の、3つの、段階を通じて、TabICLの、平均順位は、**11.4位(9位)から、7.46(2位)、最終的に、6.95(1位)**へ、改善する。しかし、私たちは、カリキュラム学習の、間、いくつかの、小さな、データセットで、**わずかな、性能の、低下も、観察する(図10)。****

(ポイント)

図5の、読み方

- 横軸:MLPに対する、**相対的な、正解率の、改善(-10%~+10%)**。
- 縦軸:各、手法(TabICLが、一番、上)。
- 箱ひげ図:各、手法の、**正解率の、分布。赤い、線が、中央値。**
- 色:**学習+推論の、時間(黄色=速い、紫=遅い)。**

図6=速度向上の、規模、依存性

- 横軸:セルの、数(行×列)。縦軸:速度向上(倍率)。
- 点が、右上に、いくほど、セルの数が、多いほど、速度向上も、大きい、という、傾向が、見えます。
- 点線の、傾きが、**この、傾向を、要約しています。**

本文の、具体例(10,000サンプル、100特徴量で、20秒、対、1分40秒)を、確認します。20秒×5=100秒(1分40秒)なので、**約5倍**、これは、本文の、「大きな、データセットで、約5倍」という、記述と、一致します。

図7=順位の、変化(サンプル数が、増えると、どうなるか)

- 横軸:サンプル数(対数)。縦軸:順位(低いほど、良い)。
- ****オレンジの、線(TabPFNV2)**は、サンプル数が、増えるほど、順位が、悪化(上に、伸びる)しています。これは、本文の、「30,000サンプルを、超えると、失敗する」の、視覚的な、証拠です。**
- ****青の、線(TabICL)**は、サンプル数が、増えても、順位が、あまり、悪化しない(比較的、平ら)。**

6. Conclusion

6. 結論

(本文)

私たちは、TabICLを、導入する。これは、**既存の、表形式基盤モデルの、拡張性を、1桁、拡張する、新しい、表形式基盤モデルである。最大、100,000の、訓練サンプルを持つ、データセットで、評価され、ハイパーパラメータ調整なしに、優れた、性能を、届ける。これにより、調整を、必要とする、他の、表形式手法より、ほぼ、2桁、速くなる。**TabICLは、これを、**ハイブリッド・アーキテクチャと、メモリ節約の、最適化を通じて、達成する。新しく、公開され、主導的な、表形式基盤モデルである、TabPFNV2と比べて、TabICLは、同等の、性能を、達成しながら、より、拡張性が、高く、速い。**

制限

(1) 他の、基盤モデルと、同様に、TabICLは、遅い、推論速度に、苦しむが、TabPFNV2は、キャッシングが、これを、緩和できることを、示した。現在、TabICLは、分類に、限られるが、回帰は、Hollmann et al. (2025)が、示したような、似た、方法論で、扱える。(2) 表形式データの、決定的な、特徴は、列に、自然な、順序が、ないことである。理想的には、表形式手法は、列の、並べ替えに対して、不変であるべきだが、TabICLは、RoPEの、使用のために、この、不変性を、破る。(3) 私たちの、評価は、TALENTベンチマークの、強みと、弱みを、引き継いでいる。特に、TALENTは、ホールドアウト法で、調整した、単一の、モデルを、訓練するが、交差検証で、評価した、モデルを、アンサンブルすることで、性能が、改善しうる。バギングは、計算コストを、増やすが、TabICLは、検証セットを、必要とせず、全データで、学習できる、というだけである。(4) TALENTは、欠損値の、扱いに、平均値による、補完を、使う。しかし、TALENTの、欠損値の、割合は、低いにも、関わらず、TabPFNV2に、欠損値を、内部的に、扱わせるほうが、良いかもしれない。(5) 私たちは、TabPFNV2に、カテゴリの、情報を、提供しなかった。TabPFNV2は、カテゴリ特徴量を、内部的に、推測するが、この、情報を、明示的に、提供すれば、性能が、高まる、可能性がある。

展望

表形式データのための、文脈内学習は、元々、小さな、表のための、高速化として、導入された(Hollmann et al., 2022)。Transformerの、1回の、順伝播の、表現力は、限られているように、見えるかもしれないので、十分な、データがあれば、文脈内学習は、その、優位性を、失うのでは、ないか、と、疑問に、思うかもしれない。私たちは、大きな、データでも、事前学習が、暗黙の、事前分布を、誘発し、それが、文脈内Transformerに、競争力のある、優位性を、与えることを、見出した。

abPFN(v1)→TabPFNV2→TabICL→TabFM(ブログ)の、全体像

「表データの、機械学習は、長年、GBDT(勾配ブースティング決定木)が、主流だったが、TabPFN(v1, 2023)が、"事前に、大量の、人工データで、学習しておき、新しい表には、1回の、推論だけで、答える"という、新しい方式を、示した。ただし、v1は、小規模データ(1,000行)、分類のみ、かつ、表の、行・列構造を、うまく、使えていなかった。TabPFNV2(2025, Nature)は、セル単位で、行・列、両方向のattentionを、導入し、10,000行、回帰にも、対応した。しかし、その、"2方向を、何度も、繰り返す"設計は、計算コストが、高く、大規模データに、弱かった。TabICL(2025, ICML)は、"まず、列を見て、行を圧縮し、そのあとで、ICLを行う"という、2段構成に、変えることで、計算量を、大幅に、減らし、50万行まで、扱えるように、した(ただし、回帰は、まだ、対応していない)。Googleの、TabFMは、この、v2の、表現力と、TabICLの、効率性を、組み合わせ、回帰にも、対応した、最新のモデルだと、考えられる。」

【出発点】表データの分類は、GBDT(木ベース)が、支配的

↳ 課題:新しい表が来るたびに、学習し直す、手間

【TabPFN v1】事前学習+ICLで、"1回の順伝播" を実現

↳ 課題:①表構造(行・列)を使えていない、②役に立たない特徴量・カテゴリ・欠損に弱い、③分類のみ、④小規模(1,000行)まで

【TabPFN v2(Nature)】セル単位+行列2方向attentionで、①②③④を解決

↳ 課題:2方向attentionを12層、交互に行うのでコストが高い(大規模データに弱い)

【TabICL】列⇒行(1回)で圧縮⇒ICL、という、2段構成でコストを削減

↳ 課題:回帰に未対応(v2で対応していたのに、後退)

【TabFM(Google、ブログのみ)】v2の表現力+TabICLの効率性を統合。回帰にも対応